

CASA Evidence — Protection from Insecure Direct Object Reference (IDOR)

Question: *Provide a written description of how the APIs are protected from Insecure Direct Object Reference attacks.*

Application: Provenance (Next.js App Router monorepo, Supabase Postgres + Auth + Storage, plus a standalone public partner REST API under `apps/api`).

This document is a companion to the three prior CASA evidence documents (`authentication-and-access-control.md`, `principle-of-least-privilege.md`, `api-user-controlled-input-inventory.md`). Those documents establish *what* identifiers users can pass in and *what* RLS/permission model exists; this document explains specifically how the codebase prevents a signed-in user from supplying **someone else's** object identifier (artwork ID, invoice ID, profile ID, exhibition ID, claim token, etc.) and having the application act on it.

An Insecure Direct Object Reference (IDOR) vulnerability exists when an application uses a client-supplied identifier to fetch or modify a resource ****without independently verifying that the caller is authorized for that specific object****. Provenance defends against this with four overlapping controls, described in order from "hardest to bypass" to "most specific":

- 1 Non-enumerable, unguessable identifiers
 - 2 Database-enforced ownership (Row Level Security) — the authoritative control
 - 1 Explicit application-level ownership re-checks — defense-in-depth, especially where RLS is intentionally bypassed
 - 1 Token-based access with additional, independent authorization checks after redemption
-

1. Non-enumerable, unguessable identifiers

The first line of defense against IDOR is making the identifier itself impractical to guess or enumerate, so that even a missing authorization check would require the attacker to already know a valid ID.

1 ****Every primary key in the schema is a random UUID**
(`uuid_generate_v4()`)****** — `accounts.id`, `artworks.id`, `user_profiles.id`, `gallery_members.id`, `exhibitions.id`, `invoices.id`, `consignments.id`, `loans.id`, etc. There are no sequential integer IDs anywhere in the object model that a client could increment to enumerate other users' resources.

1 ****Invite / claim links use a separate, cryptographically random 256-bit token — not the underlying object's ID:****

```
export function generateClaimToken(): string {
  return randomBytes(32).toString('base64url');
}

export function hashClaimToken(token: string): string {
  return createHash('sha256').update(token, 'utf8').digest('hex');
}
```

The raw token is emailed to the invitee and **never stored** in the database — only its SHA-256 hash is persisted (`certificate_claim_invites.token_hash`). Redemption looks up the invite by the hash of the token supplied, so even a full database read of the `certificate_claim_invites` table does not reveal usable tokens, and the token space (2^{256}) makes brute-force guessing infeasible.

This layer alone is not sufficient — it only raises the cost of a blind attack. The layers below are what actually enforce authorization once an ID (guessed or legitimately obtained) is presented.

2. Database-enforced ownership (Row Level Security) — the authoritative control

As documented in `authentication-and-access-control.md` §3.2 and `principle-of-least-privilege.md` §2, every table a normal request can reach has Row Level Security enabled (44 migrations, 229 policies), and the standard policy shape is:

```
create policy artworks_read_own on public.artworks
  for select to authenticated
  using (account_id = (select auth.uid()));
```

This is the IDOR control that holds **regardless of what the application code does**: because normal requests run as the Postgres authenticated role using the caller's own JWT (not a superuser), the database itself refuses to return or modify a row whose owner column doesn't match `auth.uid()` — even if a route handler forgot to add a `.eq('account_id', user.id)` filter, or even if a client tampers with a hidden form field to submit another user's ID. A request for `/api/operations/invoices/<someone-else's-invoice-id>/pdf`, for example, is blocked by RLS on the `invoices` table even before the application's own ownership check (see §3) would have caught it. This is why RLS is called the *authoritative* IDOR control in this codebase: application-layer checks (§3) are important defense-in-depth, but RLS is the backstop that holds even when they are missing, buggy, or bypassed by a code change.

3. Explicit application-level ownership re-checks

RLS only protects requests made with the caller's own session-bound client. Two situations exist where the application **must** perform its own explicit ownership check because RLS cannot (or does not) cover it:

3.1 Code paths using the service-role (admin) client

Several flows intentionally use `getSupabaseServerAdminClient()` — which bypasses RLS entirely — to perform steps that legitimately need to read/write across ownership boundaries (e.g. inviting someone *else* to claim a certificate). In every such flow, the code re-implements the ownership check by hand instead of relying on the database:

```
const { data: artwork, error: artError } = await asUntyped(client)
  .from('artworks')
  .select('id, account_id, title, certificate_type')
  .eq('id', artworkId)
  .single();

if (artError || !artwork) {
  return { success: false, error: 'Certificate not found' };
}

if (artwork.account_id !== user.id) {
  return { success: false, error: 'Only the certificate owner can invite an owner' };
}
```

This lookup runs on the **session-bound** client (so RLS still applies to the read itself), but the explicit `account_id !== user.id` comparison is what actually stops a user from inviting someone to claim co-ownership of an artwork they don't own — belt-and-suspenders on top of RLS, and the only check at all for the subsequent admin-client writes (rate-limit counters, invite row insert) that follow in the same function.

`src/app/operations/_actions/consignments.ts` follows the identical pattern (`assertArtworkOwned()`, documented in `authentication-and-access-control.md §3.3`), re-verifying that `artworks.account_id === userId` before allowing a consignment to be created against a client-supplied `artwork_id`.

3.2 Multi-party ownership (shared/collaborative resources)

Some resources can legitimately be modified by more than one identity — e.g. a gallery profile by its owner **or** a team member. Rather than only checking "is this literally my row," the code resolves the full authorization relationship before allowing the

write:

```
const { data: profile, error: fetchError } = await sb
  .from('user_profiles')
  .select('user_id, role, slug')
  .eq('id', input.profileId)
  .single();

if (fetchError || !profile) {
  return { error: 'Profile not found' };
}

const isOwner = profile.user_id === user.id;
const isGalleryTeamMember =
  profile.role === USER_ROLES.GALLERY &&
  (await isGalleryMember(user.id, input.profileId));

if (!isOwner && !isGalleryTeamMember) {
  return { error: 'You do not have permission to update this profile' };
}
```

A client-supplied profileId cannot be used to edit an arbitrary profile — it is only accepted if the caller is the profile's own user_id **or** a resolved team member of that specific gallery profile, checked against the gallery_members table rather than trusted from the request.

3.3 Explicit caller-vs-target-ID comparison in query-parameter routes

For the handful of GET routes whose *only* input is an ID passed as

a query parameter (see `api-user-controlled-input-inventory.md §2.2`),

the code does a direct, explicit comparison before running any query

at all:

```
// Verify the user is authenticated and requesting their own exhibitions
const client = getSupabaseServerClient();
const { data: { user } } = await client.auth.getUser();

if (!user) {
  return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
}

// Users can only get their own exhibitions
if (user.id !== userId) {
  return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
}
```

`/api/check-exhibition-ownership` goes a step further: it checks

`user.id !== userId` (rejecting an obviously spoofed caller identity)

and then separately re-derives ownership from the database

(`exhibitions.gallery_id === user.id`) rather than trusting the

client's claim of ownership at all — the client-supplied `userId` is only ever used to confirm "this is who you say you are," never as the basis for an authorization decision by itself.

3.4 Avoiding existence-leak (404 vs. 403)

`/api/operations/invoices/[id]/pdf`, `/loans/[id]/pdf`, and `/consignments/[id]/pdf` all combine the ownership filter directly into the lookup query rather than fetching-then-checking:

```
const { data: inv, error: invErr } = await asUntyped(client)
  .from('invoices')
  .select(...)
  .eq('id', id)
  .eq('account_id', user.id)
  .maybeSingle();

if (invErr || !inv) {
  return NextResponse.json({ error: 'Not found' }, { status: 404 });
}
```

Because the `account_id` filter is part of the same query, a request for another user's invoice ID returns an identical 404 `Not found` — the same response as a nonexistent ID. This avoids leaking *which* IDs exist at all (an oracle that 403 `Forbidden` would otherwise provide), which is itself a defense against ID-enumeration attacks that often precede a full IDOR exploit.

4. Token-based access with independent post-redemption checks

Certificate claim/invite links (`/claim/certificate?token=...`) are a special case: the "object reference" in the URL is the random token from §1, not a database row ID, and the resource it unlocks (co-ownership of a certificate) is inherently meant to be redeemable by someone who is *not* yet the row's owner. IDOR protection here means: possession of the token must not be the *only* check.

`consumeCertificateClaim()` layers three independent checks after the token hash is matched, in order:

```
const tokenHash = hashClaimToken(trimmed);
const adminClient = getSupabaseServerAdminClient();
```

```
const { data: invites, error: inviteError } = await asUntyped(adminClient)
  .from('certificate_claim_invites')
  .select('*')
  .eq('token_hash', tokenHash);

if (inviteError || !invites?.length) {
  return { success: false, error: 'Invalid or expired claim link' };
}
```

1 **Email match** (`validateInviteeEmail`) — the signed-in user's authenticated email must match the invite's `invitee_email`; a token forwarded to (or intercepted by) a different signed-in account is rejected even though the token itself is valid.

1 **Expiry check** (`validateExpiry`) — invites older than the 14-day TTL are rejected and flipped to expired server-side, regardless of token validity.

1 **Status / role check** — the invite must be in a redeemable state (sent/pending, not already consumed/cancelled), and for gallery/artist claim kinds the signed-in account's role is independently re-derived from `accounts.public_data` (never trusted from client input) before the claim is allowed to proceed. This means a leaked or forwarded claim link is not, by itself, a usable "direct object reference" to someone else's certificate — it only works for the specific invited email address, within the expiry window, in the correct status, for an account with the correct role.

5. Public partner API (`apps/api/v1`) — API-key scoping in place of session ownership

The standalone partner API (documented fully in `api-user-controlled-input-inventory.md §1`) has no user session at all — the "caller identity" is the API key. IDOR protection here is enforced by binding every request's path parameters to the authenticated key's own scope:

1 **Writes** (`POST /api/v1/assets/{planet}`) always set `account_id/created_by` from `auth.accountId` (the key's own

account) **after** spreading the request body — a caller cannot pass an `account_id` field in the body to create an asset under a different account.

1 Planet-scoped keys are rejected by `requirePlanet()` if the path's `{planet}` segment doesn't match the key's own scope, so a key issued for one vertical cannot be pointed at another vertical's assets by simply changing the URL.

1 Reads by ID (`GET /api/v1/assets/{planet}/{id}`) are intentionally a curated *verification* lookup, not a general "get my private data" endpoint: they require the `verify` scope, and the response uses an explicit column allow-list (`PUBLIC_ASSET_COLS`) that excludes anything not meant to be publicly checkable — the same design already used for anon-readable rows in the main app (status = 'verified' artworks, published exhibitions, etc., see `principle-of-least-privilege.md §2`). Being able to look up asset `{id}` by ID is the intended product behavior (analogous to checking a VIN or a certificate number), not an authorization gap — provided the response never exposes non-public fields, which the column allow-list guarantees.

6. Summary — control by resource type

Resource / access pattern	Non-enumerable ID?	RLS-enforced?	Explicit app-level re-check?		
Artworks, invoices, loans, consignments, exhibitions, profiles (owner CRUD)	Yes (UUID)	Yes — <code>account_id/user_id = auth.uid()</code>	Yes, in every Server Action/route that also uses the admin client or accepts an ID as a bare parameter		
Gallery team-shared resources	Yes (UUID)	Yes, plus role-aware policies (owner/admin/member)	Yes — <code>`isOwner`</code>	<code>`</code>	<code>isGalleryTeamMember`</code> resolved server-side

Resource / access pattern	Non-enumerable ID?	RLS-enforced?	Explicit app-level re-check?		
Certificate claim / owner-invite links	Yes — random 256-bit token, hashed at rest	N/A (admin-client flow by design)	Yes — email match, expiry, status, and role re-checked independently after token match		
Query-param "is this my resource" routes (get-user-exhibitions, check-exhibition-owner-ship)	Yes (UUID)	Yes (underlying table)	Yes — explicit <code>callerId === paramId</code> comparison, plus a second DB-derived ownership check		
PDF export routes (invoices/loans/consignments/[id]/pdf)	Yes (UUID)	Yes	Yes — ownership folded into the query itself; mismatch returns 404, not 403, to avoid existence-leak		
Public partner API asset/certificate lookups	Yes (UUID / certificate number)	N/A — service-role client, by design	Yes — API-key scope/planet binding for writes; reads limited to an explicit public-safe column allow-list		

7. Conclusion

IDOR protection in this codebase is not a single check but a layered system: identifiers are unguessable by default (UUIDs, and separately random hashed tokens for share links), the database independently enforces per-row ownership for every table via RLS regardless of what the application code does, and every code path that legitimately needs to look up a resource by a client-supplied ID — especially those using the RLS-bypassing service-role client — adds its own explicit ownership/authorization check rather than trusting the identifier alone. The public partner API achieves the same outcome through API-key scope binding instead of session ownership, since it has no authenticated user session to compare against.